

Software Requirements Specification

Requirements

Stacks is a web-based social reading platform designed to help users track their reading journey, discover books at local California libraries, and receive personalized recommendations. The system combines personal library management with community features, barcode scanning technology, AI powered recommendations, and integration with 50+ California public library systems.

The system consists of four primary components:

1. User facing web interface (React)
2. User authentication and profile management (Firebase Auth)
3. Book discovery and search layer (Open Library API)
4. Structured data store containing user shelves, books, and reviews (Cloud Firestore)
5. AI recommendation engine (Google Gemini API)
6. External library catalog integration (direct links to library systems)

High-level component diagram:

- User → Web Interface (React)
- Web Interface → Firebase Auth (Authentication)
- Web Interface → Cloud Firestore (User Data, Shelves, Reviews)
- Web Interface → Open Library API (Book Metadata)
- Web Interface → Gemini AI (Recommendations)
- Web Interface → Library Catalog Systems (via customSearchPattern links)
- Web Interface → Camera API → BarcodeDetector (ISBN Scanning)

Functional Requirements

This section describes the features and capabilities of the Stacks application. Requirements are organized by major system functionality and specify what the system shall do without detailing implementation specifics.

User Access and Navigation

The system shall allow users to create an account using email and password.

The system shall allow users to sign in using email and password authentication.

The system shall allow users to sign out of their account.

The system shall enforce unique usernames across all users.

- Usernames will be between 3 and 30 characters in length.
- Usernames will contain only alphanumeric characters and underscores.

The system shall allow users to update their profile information including display name, username, email, and bio.

- Bio text will be limited to 500 characters.

The system shall validate email format before allowing account creation or email updates.

The system shall provide error feedback for authentication failures including invalid credentials, email already in use, and username already taken.

Book Search and Discovery

The system enables users to search for and discover books using multiple methods.

The system shall allow users to search for books by title using text input.

The system shall return search results from the Open Library API matching the query.

The system shall display search results as visual cards showing book cover, title, author, and publication year.

The system shall allow users to scan book barcodes using their device camera to search by ISBN.

The system shall store custom book entries in Firestore when a scanned book is not found in the Open Library API.

- Custom entries will be searchable by all users in future searches.

The system shall display detailed book information including description, author(s), publisher, publication date, page count, and ISBN when available.

Personal Library Management (Shelves)

The system allows users to organize books into customizable shelves with smart categorization.

The system shall create three default shelves for each new user: "Want to Read", "Currently Reading", and "Read".

The system shall allow users to add books to any shelf.

The system shall allow users to remove books from any shelf.

The system shall enforce exclusive placement on default shelves (a book can only be on one default shelf at a time).

- When a book is added to a default shelf, it will be automatically removed from other default shelves.

- Custom shelves will not have this restriction.

The system shall allow users to create custom shelves with user-defined names.

The system shall allow users to delete custom shelves.

- Default shelves cannot be deleted.

The system shall display shelf contents as a grid of book covers with title and author information.

The system shall indicate which shelf(s) contain a book when viewing book details.

- The shelf button matching the current default shelf will display in a distinct color (forest green).

The system shall provide a shelf selector modal allowing users to toggle book presence across multiple shelves simultaneously.

- The modal will display checkboxes for all user shelves.
- The modal will allow inline creation of new shelves.

Ratings and Reviews

The system enables users to rate and review books they have read.

The system shall allow users to rate books on a scale of 0 to 5 stars in half-star increments.

The system shall allow users to write text reviews for books.

The system shall display user ratings and reviews on book detail pages.

The system shall allow users to edit their own ratings and reviews.

The system shall allow users to delete their own ratings and reviews.

The system shall calculate and display average ratings across all user reviews for each book.

The system shall display the total count of reviews for each book.

The system shall prompt users to rate and review books when marking them as finished.

- This prompt will appear when moving a book to the "Read" shelf.

Library Discovery and Integration

The system helps users find books at nearby California public libraries.

The system shall allow users to search for libraries by entering a location (city, zip code, or address).

The system shall display a list of nearby libraries within a 15-mile radius of the specified location.

The system shall display library information including name, address, distance from user, and phone number.

The system shall provide clickable links that open the library's online catalog with the current book pre-searched.

- Links will use library-specific URL patterns (customSearchPattern) appropriate to each library's catalog system (BiblioCommons, SirsiDynix, Polaris, etc.).

The system shall encode book titles and ISBNs properly in library catalog URLs using URL encoding.

AI-Powered Recommendations (Stack Match)

The system provides personalized book recommendations using artificial intelligence.

The system shall allow users to request book recommendations from their Want to Read shelf.

The system shall accept user input for preference filters including:

- Genre preference
- Mood/vibe description
- Trope preference
- Micro-trope or specific scene description

The system shall generate personalized recommendations using the Google Gemini API.

- Recommendations will be based on user preferences when provided.

- When no preferences are specified, recommendations will be based on the user's 4+ star rated books from reading history.

The system shall display 3 recommended books in a card based interface.

- Each card will show book cover, title, author, and AI-generated reasoning for the recommendation.

The system shall allow users to accept or pass on each recommendation.

- Accepting moves the book to a prioritized position in the user's reading queue.

- Passing removes it from the current recommendation set.

The system shall display appropriate messages when the Want to Read shelf is empty or no suitable recommendations can be generated.

Library Savings Tracker

The system calculates and displays money saved by borrowing books from libraries.

The system shall track books that users have marked as borrowed from a library.

The system shall calculate estimated savings using an industry average book price of \$18

The system shall display the total number of books borrowed and total estimated savings on the user's profile.

The system shall update the savings calculation in real time as users log more library borrowed books.

The system shall display an encouraging message when no library books have been logged yet.

Performance Requirements

Performance requirements define how well the system operates under expected usage conditions.

Page Load Time

The system shall load any primary application page within 5 seconds under normal network conditions.

Search Response Time

The system shall return the first set of search results within 5 seconds of a user submitting a search query.

Concurrent Usage

The system shall support multiple concurrent users without data corruption or application failure.

AI Recommendation Generation Time

The system shall generate Stack Match recommendations within 10 seconds of request submission.

Environment Requirements

Development Environment Requirements

- A modern code editor or integrated development environment
- Access to a version control system
- Internet access for testing external library links

Execution Environment Requirements

Hardware Requirements:

- Any device capable of running a modern web browser

Software Requirements:

- A web browser (Chrome or Safari)
- Internet connectivity

The system does not require specialized hardware or proprietary software for execution.

Notes

Library Catalog Integration:

The system does not perform checkout or reservation transactions. All library interactions redirect users to official library catalog systems where they can check availability and place holds using their library card credentials.

AI Recommendations:

The quality of AI generated recommendations depends on the accuracy of book metadata and the specificity of user preferences. The system uses the Gemini 3 Flash model for optimal speed and rate limits on the free tier.